

Implementations of the State Merging Algorithm for DFA Learning

Progress Report

Matthew Axisa
University of Malta
matthew.axisa.18@um.edu.mt

ABSTRACT

Grammatical inference is the process of learning a formal grammar from 2 sets of strings which do and do not belong to the language. Typically, inference algorithms rely on a process of state merging which iteratively combines pairs of compatible states in an automaton to create a smaller, more general automaton. This process is repeated a large number of times many times during the inference process. In this research we will investigate the properties of state merging, attempting to find a more efficient implementation of the merging operation. Computational complexity of potential state merging operations will be examined before they are empirically tested against state of the art techniques.

1. INTRODUCTION

1.1 Problem Definition

The problem of grammatical inference is transversal to numerous fields in computer science. Broadly speaking, the field of study focuses on creating a representation which can define rules for a set of structured data. Depending on the specific problem, the target automaton should accurately classify, generate or otherwise manipulate data. One of the best studied grammatical inference problems is that of deterministic finite state automaton (DFA) learning where a set of positive and negative examples are given and the task is to create the smallest possible DFA consistent with the sample.

Learning grammars is difficult; one of the earliest theoretical results from Gold established that the problem of learning Chomsky Type-3 grammars is NP-Complete [1] even without noise in the training data. Despite this, since the first *International Conference on Grammatical Inference* in 1994 there have been a steady stream of improvements in the field, including by not limited to new algorithms such as RPNI [2] and EDSM [3] and improvements on the latter heuristic [4–6] as well as SAGE [7] and heuristic guided search [8]. More recently, other researchers have used SAT [9] solvers or SMT solvers [10] to tackle the problem.

Common to many of the aforementioned DFA learning algorithms is the state merging operation which combines two states in a DFA to produce a smaller DFA which accepts more strings. At each step in the algorithm there are a huge number of possible merges and for all but the smallest DFA learning problems, learning a grammar will require literally millions of state merging operations. The goal of heuristics such as EDSM is to choose the next merge and this is done by computing a score for each merge. This approach means

that at every step, each merge must be computed. Thus, the speed at which a solution to a problem can be found is directly related to the speed of the state merging operation. The blue-fringe framework [3] and windowed EDSM [11] reduce the number of merges required for each step by intelligently restricting the number of merges considered. While this does greatly improve speed since some of the search space is being disregarded the techniques are less accurate than original EDSM [3].

1.2 Motivation

The need for a faster state merging algorithm is motivated by the sheer number of state merges that DFA learning algorithms perform. Between the PTA (see section 3) and the output of the algorithm a large problem may require hundreds of millions of merge operations to be performed. The massive quantity of merges provides ample motivation this research since even fractions of a second make a difference over such a large number of operations. Dupont et al. elaborate on the search space of the the DFA learning problem [12] and their work is discussed further in section 4. However, needless to say the search space is very large this is only compounded by those algorithms which backtrack, exploring more than one path in the search space from the initial hypothesis.

Additional motivation for a faster state merging operation comes from real-world applications. Map learning for robotics and control systems are another possible application for DFA learning. Examples of work in this field are from Reiger and An [13] who infer an automaton from sensor data; because of noise in the sensor, this application uses a probabilistic automaton. Rivest and Schapire [14] use of unique and very long homing sequences to infer an unknown DFA from its behavior; this application is interesting because it provides no way to arbitrarily reset the automaton to its start state and also the inference process requires a teacher to provide counter-examples. For this technique to see applications beyond simply research, the algorithms for DFA learning it employs should be fast enough to run with hardware limitations often seen in robotics; a fast state merging operation is paramount for this application.

1.3 Why the Problem is Nontrivial

When merging two states from an automaton usually determinism is not preserved. Determinism can be restored by a recursive process, however the process visits many of the states in the DFA ensuring that their labels are compatible. Especially during the first few merges where large

parts of the DFA are still tree-like, merging any two states may require a large number of states to be visited and evaluated. Additionally, every time two states are merged this may cause previously valid merges to become invalid meaning that at each time step the learning algorithm must re-evaluate merges to ensure they are valid. The problem is non-trivial because of the way an arbitrarily small change to a part of the automaton gives cause to question the validity of seemingly unrelated merges. No criteria exist which can be used to decide whether a merge remains valid after a change to the automaton.

The prefix tree acceptor (PTA) which is the starting point for EDSM and RPNI has orders of magnitude more states than the target DFA. While each potential merge is computed one by one meaning that the amount of memory required at a given moment is relatively small, copying the original DFA for each operation is inefficient because of the sheer number of potential merges. An alternative to this is to use a journaling data structure which can rollback to its previous state after each merge, however this implies an overhead to track the changes made by each merge. Part of the difficulty of this problem is defined by this duality between copying data and recording changes and current data structures are inadequate for solving this problem.

2. AIMS AND OBJECTIVES

The goal of this project is to produce a state merging operation which will significantly speed up state of the art DFA learning algorithms. In order to archive this, the following objectives have been identified:

- research current implementations of state merging operations and identify their drawbacks
- investigate and design a data structure suited to the state merging problem
- generate a statistically significantly sized pool of suitable testing problems and quantitatively measure the speed up produced by the new state merging operation
- compare new state merging implementations with state of the art implementations

3. BACKGROUND

A deterministic finite state automaton (DFA) is a 5 tuple $(Q, \Sigma, \delta, q_0, F)$ which are the set of states, alphabet, transition function, starting state and set of final states respectively. An automaton can process strings constructed from letters in the alphabet by starting from q_0 and transitioning to other states in Q according to δ . A string is either accepted or rejected by the automaton depending on whether the final state it lands on is an element of F . For the purpose of studying these automata the alphabet is restricted to have only 2 characters; for real-world applications, higher order alphabets can be mapped to a binary alphabet.

In DFA learning a training set of positive and negative strings are given to a learning algorithm whose task is to reconstruct the original DFA that classified those strings. The training set of strings will affect how difficult it is to learn the target DFA. Primarily and intuitively the more strings the training set contains the easier the learning task will be. Apart from this one important definition is that of structural

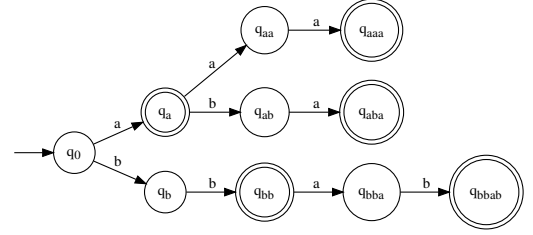


Figure 1: The PTA for a dataset with positive examples $\{a, aaa, aba, bb, bbab\}$.

completeness; for a training set to be structurally complete it must contain strings which exercise every transition in the automaton. Even with a perfect learning algorithm it may be impossible to learn the correct DFA if the given dataset is not structurally complete; there simply would not be enough information to define some transitions.

Typically, state merging algorithms begin from a *prefix tree acceptor* (PTA). The PTA is a DFA which is structured like a tree and is built using the positive strings in the dataset. For each string a path from the start state using characters in that string is created and the last state is marked as final, states can be reused if two strings have a common prefix. The PTA is the most specific DFA which is strongly consistent with the sample, meaning it respectively accepts and rejects all positive and negative strings in the dataset. From the PTA, states are merged to create a more general DFA; which states are merged is decided by the learning algorithm.

4. LITERATURE REVIEW

The origins of this field of study can be traced back to three fields [15]. In his work related to computational linguistics, Chomsky first introduced the key notions for formal language theory and posed questions about discovering a semantic basis for grammars [16]. Solomonoff did preliminary research into inductive inference [17] and then formalized this theory in a later work [18]. The field of structural and syntactic pattern recognition used grammars extracted from structured data [19, 20]. This is perhaps the oldest application of GI and Miclet even directly references the formal language theory developed by Chomsky and names the problem *Grammatical Inference* in his contribution to [21]. The question of learning grammars from a set of examples is general enough that it has been tangential to a plethora of other research fields, however it is also specific enough that grammatical inference has become a decidedly distinct research area.

It is widely accepted that the first theoretical contribution to the area comes from Gold who defined identification in the limit and showed that regular languages (which can be represented with DFAs) had this property. In a nutshell, a class of languages is said to be *identifiable in the limit* if, for any unknown language L in that class, only a finite set of training examples from that language need to be given before the learner will exactly produce the language. After this, the learner will never produce another language

even when presented with a larger training set [22]. In addition to this, Gold also consolidated the difficulty of this problem by showing that the question ‘*is there an automaton with n states which agrees with D* ’ is NP-Complete [1] - where D is a finite set of observations of the I/O behavior of that automaton. It should be noted that in this definition, the learner never has to definitively state that the automaton created is the final automaton, just that at some point the automaton produced will exactly match the target. The main problem with Gold’s definition is that there is no bound on the time taken for the learner to exactly predict the target; the number of strings required could be exponential or super-exponential in the size of the target automaton. An important expansion on this definition came from Pitt [23] who defined *polynomial identification in the limit*, an important definition which set complexity bounds for the update time and number of mistakes an algorithm can make.

Dupont et al. posed the question of DFA learning as a search through the automata in the space between the prefix tree acceptor (PTA) and the universal automaton [12]. To summarize, the paper expands on the definition of structural completeness saying that for a training set to be structurally complete with respect to a DFA it must not only contain strings which exercise all of the DFA’s transitions but also the strings should exercise each of its final states. The paper defines a partial order over partitions of states and uses their extended definition of structural completeness to show that if a training set is structurally complete with respect to a target DFA, then that DFA will necessarily exist in the search space between the PTA and the universal automaton. Additionally, the target automaton belongs to a set of automata in the search space which Dupont et al. coined the border set; this is the set of the most general automata in the search space which are still consistent with the training examples. Dupont et al. use the phrase “generalisation as search” [12] to describe DFA learning algorithms which is exactly the process of state merging algorithms which begin with the PTA (the most specific hypothesis) and reach a final automaton by generalizing the automaton through state merging.

Despite the fact that results from Gold [1] show that DFA learning is difficult, there are specific criteria which, when fulfilled, can make the problem solvable in polynomial time. Trakhtenbrot and Barzdin were the first to use a state merging algorithm which is the fundamental idea for many more recent algorithms. They showed that given a training set complete with all strings up to a given length l , then a state merging algorithm is guaranteed to find the smallest consistent DFA. The length l depends on the *degree of distinguishability* of the target automaton and its depth in the sense defined in [24] and in the worst case $l = 2n - 1$. Of course a set of all strings up to some length l is maximally dense and contains 2^l elements making the problem intractable for large values of l . Lang empirically showed that fortunately with only a subset of this training data an approximately correct hypothesis can often be found [25].

In his original 1978 paper, Gold described an algorithm for DFA learning. The algorithm was the first algorithm for DFA learning with strong convergence properties (when given adequate training data) but in many cases the algorithm would fail, when this happened it could do no better than simply return the PTA. Independently of Lang, in

1992 Oncina and García published their research detailing a new algorithm RPNI (Regular Positive and Negative Inference) [2]. This algorithm was the first of its kind in many ways, primarily it was the first example to use both positive and negative training examples, it is also the first algorithm which always produces a DFA compatible with the training sample. While not necessarily minimal, in all but the rarest of cases the automaton produced is significantly smaller than the PTA. This is because RPNI learns incrementally by state merging; after each merge a smaller DFA (still consistent with the training sample) is produced, once there are no more potential merges the algorithm terminates. This algorithm also has the property of identifying in the limit a DFA when given a characteristic sample [15].

Catalysts for the development of new algorithms for DFA learning were the two competitions Abbadingo One¹ and Gowachin². Abbadingo One was organised by Lang and Perlmutter in an attempt to promote the development of algorithms in the field, they focused on gathering empirical data based on problems which were harder than those attempted by learning algorithms developed in the past with particular focus on sparser training sets [3]. The competition was a success and the winning algorithm was Rodney Price’s EDSM (Evidence Drive State Merging). The experimental setup was concerned with learning DFAs which had 64, 128, 256 or 512 states using training sets at 4 different densities. The number of states n for a given DFA was actually its *nominal* size; the actual size of the DFA could be slightly larger or slightly smaller than that. A valid automaton for Abbadingo One had a depth of exactly $2\log_2 n - 2$. Training sets were comprised of strings drawn at random and without replacement from the set of all strings with lengths between 0 and $2\log_2 n + 3$ inclusive. A problem was considered solved when at least 99% of the testing set comprised of 1800 strings were labelled correctly. A couple of notable properties about the Abbadingo problems are that they are not guaranteed to be structurally complete and they are the same type of DFA studied by Lang regarding Trakhtenbrot and Barzdin’s algorithm in [25]. Gowachin was set up after the Abbadingo One competition was over and aimed to provide a benchmark for algorithms for DFA learning algorithms. The setup here is slightly different as participants could generate problems on demand and choose parameters such as the size of the DFA and the number of sample strings. Apart from this, the competition also allowed for participants to introduce noise into the training set. The contest closed in 2004, however there is still a hall of fame available on the Gowachin website. Both competitions eliminated the possibility of hill-climbing algorithms. Abbadingo One and Gowachin are important to mention because the design of their problems are useful benchmarks for DFA learning and some of the problems proposed in the Abbadingo One competition remain unsolved to this day.

EDSM is based on the same state merging principals as RPNI. The method starts with the PTA which is built from the training data and then repeatedly merges compatible states [3]. The problem is that the sparser the training data gets, the more difficult it is to decide whether compatible merges are correct. Each merge changes the structure of the automaton and implicitly creates constraints on future

¹<http://abbadingo.cs.nuim.ie/>

²<http://www.irisa.fr/Gowachin/>

	Density			
	IV	III	II	I
64	4456	3478	2499	1521
128	13894	10723	7553	4382
256	36992	28413	19834	11255
512	115000	87500	60000	32500

Figure 2: Summary of the Abbadingo problem matrix. The number in each cell corresponds to the number of training strings provided for problems with a given DFA size and training set density. Shaded cells are those solved during the competition; problems with a density of 4 were already solvable by the Trakhtenbrot and Barzdin algorithm [24] and thus were considered training problems.

merges, an incorrect merge will lead to bad constraints. This effect is magnified the longer the algorithm runs and thus it is essential that the early merges are correctly chosen [3]. Early on in the algorithm, the DFA will still resemble a tree, traversing nodes and considering merges in breadth first order will mean that nodes with larger subtrees are considered for merging first. Lang claimed that because larger subtrees likely encode more information from the training set, considering merges in breadth first order meant that the first acceptable merge found probably had the most evidence that it is a good merge [25]. Higera et al. suggested a better strategy for scoring merges [26] by looking at training data however Lang et al. claims that their heuristic is flawed [3]. Price won the Abbadingo One competition because he suggested a better measure to score merges. After the equivalence classes are calculated the score is a sum of calculating the following for each equivalence class; if there are conflicting labels for states in the class the score is $-\infty$, if the class has no labels the score is 0, otherwise the score is the number of labels minus 1. In other words, every label which is checked and is okay awards 1 point to that merge, any label inconsistency yields a score of $-\infty$ [3]. The accepted merge is the one with the highest evidence score. The algorithm halts when all possible merges have a negative score, indicating that they are invalid. Since its inception a number of optimisations have been proposed such as using windowed EDSM or applying the blue fringe framework. These changes essentially aim to reduce the number of merges which have to be tested however in an ideal world, evidence for every merge would be calculated. EDSM remains one of the best algorithms for solving DFA learning problems.

It should be noted that for all state merging algorithms, merging two states often results in non-determinism in the automaton. Determinism can be restored using a recursive algorithm, which is part of the state merging process. This recursive process is practically the same for all state merging operations and descriptions have been given by Oncina and García [2] as well as by Lang et al. [3]. In the case of the blue-fringe framework at least one of the nodes being merged is guaranteed to be the root of a tree. A description of RPNI and EDSM in the blue-fringe framework are given

by [15] and [3] respectively.

On the topic of implementations of state merging few proposals have been made. Section 1.3 briefly touched on the problems of deciding whether the state merging data structure should be repeatedly copied or should be rolled back after each operation but these are not the only considerations. The reference implementation of EDSM [3] used a union-find data structure because it allows for near constant amortized time merging, adding and finding operations. While for a single merge these properties are ideal, the research does not discuss how the large number of merges are handled at each step. Another reference implementation of RPNI from the Stamina DFA learning competition³ uses a union-find data structure which uses journalling to enact rollbacks on merges which are rejected, however there is very little literature discussing the performance of this state merging implementation.

In 2010 Akram et al. released a GI toolkit for MATLAB-footnote<https://code.google.com/archive/p/gitoolbox/>. This toolkit developed to address skepticism regarding the use of MATLAB for GI research [27] and was kept up to date until March 2011. In their research RPNI and EDSM were implemented and tested with different problem instances with target DFA sizes ranging from 3 to 20 states and sample sizes ranging from 200 to 5000 strings - comparatively small problems. The performance of the algorithms was as expected, correlated with the density of the training set and the size of the target DFA, but the paper provides concrete results for the time in seconds for their implementation to solve problems. In addition to this Akram et al. claim that using a parallel implementation of RPNI they were able to gain a 10 to 15% speed increase for every CPU added [27]. The toolkit is published under the MIT License for open source software and upon inspection the DFA instance is copied for each merge operation. MATLAB also provides an interesting playground for DFA learning since it primarily works with matrices.

5. PROPOSED SOLUTION

The state merging operation and the algorithms which use it have been discussed in depth in the previous sections. An efficient implementation of the state merging operation is integral to the performance of state of the art DFA learning but the research concerned with this implementation is lacking. This research will endeavour to design or adapt a data structure which can be used to efficiently effect merges and calculate their score according to state of the art heuristics. The baseline solution will start with a union-find data structure.

One path which may be potentially explored is restricting merges with the rules from the blue-fringe framework. Knowing that one of the merge nodes is the root of a tree may be useful for optimizations which were previously not considered.

Time permitting, the research would investigate merging states, posed as a problem of operations on matrices. Before anything else, a valid and efficient representation of DFAs through matrices would need to be designed. Once this is complete, matrix operations could be used to implement state merging. We suspect that memory I/O will be a bottleneck for merging operations and since GPU memory is

³<http://stamina.chefbe.net/>

optimized for high speed operations on matrices we suspect that a GPU implementation could offer considerable speed gains.

6. EVALUATION PLAN

When evaluating state merging implementations, prospective solutions will first be studied from a theoretical standpoint and we will attempt to find computational bounds on the performance of the new techniques. Following this, techniques which are deemed to be passable from a theoretical standpoint will be implemented and tested empirically. The evaluation process will include generating a statistically significant sample of problem instances and benchmarking the current best techniques against ours. To ensure that evaluation is fair and the current best implementations will be rewritten and tested on the same pool of evaluation problems on the same machine using both RPNI and ESDM as well as flavours of those algorithms.

The pool of problem instances generated for testing will be of the same type as Abbadingo One. This means that problems of various sizes and training set densities will be tested. This would reveal whether different implementations perform better or worse depending on the size of the training set.

However, computation is not the only resource which will be taken into account. To ensure that the evaluation is well rounded, the research will study the space complexity of state merging implementations. As well as benchmarking the speed of each state merging implementation, benchmarking of the peak and average memory usage will be part of the evaluation process. This will ensure that any prospective implementation is suitable for modern hardware.

Finally, one more option for evaluation could be to investigate whether each technique is suitable for a parallel implementation. Our results would be compared to the 10 to 15% performance gain per CPU which Akram et al. were able to achieve [27]. As it stands, DFA learning algorithms are highly parallelizable and more CPU cores are becoming staple for even consumer electronics. Being able to use multiple threads or run multiple merges concurrently is a favorable characteristic for state merging operations.

7. CONCLUSION

This report presents the research which has been conducted so far. The report has outlined the problem of state merging, why it is important to the DFA learning algorithms and what facets of the problem make it non-trivial. Additionally, an overview of the literature related to the field of DFA learning has been given detailing state merging algorithms, current benchmarks and highlighting the lack of research into the performance of the state merging operations in isolation. With this research we aim to address this with an implementation which will be fairly evaluated next to current implementations both in terms of computational complexity and in terms of empirical performance.

References

- [1] E. Gold, "Complexity of automaton identification from given data," *Information and Control*, vol. 37, pp. 302–320, Jun. 1, 1978. DOI: 10.1016/S0019-9958(78)90562-4.
- [2] J. Oncina and P. García, "Inferring regular languages in polynomial update time," *World Scientific*, Jan. 1, 1992, ISSN: 978-981-02-0881-3. DOI: 10.1142/9789812797902_0004.
- [3] K. Lang, B. Pearlmutter, and R. Price, "Results of the abbadingo one DFA learning competition and a new evidence-driven state merging algorithm.," presented at the LNCS (LNAI), vol. 1433, Jan. 1, 1998, pp. 1–12, ISBN: 978-3-540-64776-8. DOI: 10.1007/BFb0054059.
- [4] S. Spina, *Merge interactions a new heuristic for automata learning*, 2004.
- [5] K. Lang, "Evidence driven state merging with search," Nov. 22, 1998.
- [6] B. Lambeau, C. Damas, and P. Dupont, "State-merging DFA induction algorithms with mandatory merge constraints," Sep. 22, 2008, pp. 139–153. DOI: 10.1007/978-3-540-88009-7_11.
- [7] H. Juillé and J. Pollack, "A sampling-based heuristic for tree search applied to grammar induction.," Jan. 1, 1998, pp. 776–783.
- [8] M. Bugalho and A. L. Oliveira, "Inference of regular languages using state merging algorithms with search," *Pattern Recognition, Grammatical Inference*, vol. 38, no. 9, pp. 1457–1467, Sep. 1, 2005, ISSN: 0031-3203. DOI: 10.1016/j.patcog.2004.03.027.
- [9] M. J. H. Heule and S. Verwer, "Exact DFA identification using SAT solvers," in *Grammatical Inference: Theoretical Results and Applications*, J. M. Sempere and P. García, Eds., ser. Lecture Notes in Computer Science, Berlin, Heidelberg: Springer, 2010, pp. 66–79, ISBN: 978-3-642-15488-1. DOI: 10.1007/978-3-642-15488-1_7.
- [10] R. Smetsers, "Grammatical inference as a satisfiability modulo theories problem," *arXiv:1705.10639 [cs]*, May 30, 2017. arXiv: 1705.10639.
- [11] O. Cicchello and S. C. Kremer, "Beyond EDSM," in *Proceedings of the 6th International Colloquium on Grammatical Inference: Algorithms and Applications*, ser. ICGI '02, Berlin, Heidelberg: Springer-Verlag, Sep. 23, 2002, pp. 37–48, ISBN: 978-3-540-44239-4.
- [12] P. Dupont, L. Miclet, and E. Vidal, "What is the search space of the regular inference?," Jan. 21, 2006, pp. 25–37, ISBN: 978-3-540-58473-5. DOI: 10.1007/3-540-58473-0_134.
- [13] A. An and A. Rieger, "Inferring probabilistic automata from sensor data for robot navigation," Mar. 8, 1999.
- [14] R. L. Rivest and R. E. Schapire, "Inference of finite automata using homing sequences," in *Proceedings of the twenty-first annual ACM symposium on Theory of computing*, ser. STOC '89, New York, NY, USA: Association for Computing Machinery, Feb. 1, 1989, pp. 411–420, ISBN: 978-0-89791-307-2. DOI: 10.1145/73007.73047.
- [15] C. de la Higuera, *Grammatical Inference: Learning Automata and Grammars*. Cambridge: Cambridge University Press, 2010, ISBN: 978-0-521-76316-5. DOI: 10.1017/CB09781139194655.
- [16] N. Chomsky, "Grammatical inference," in *The Logical Structure of Linguistic Theory*, Springer US, 1975, p. 45, ISBN: 978-0-306-30760-7.
- [17] R. J. Solomonoff, "A preliminary report on a general theory of inductive inference," 1960.
- [18] —, "A formal theory of inductive inference. part i," *Information and Control*, vol. 7, no. 1, pp. 1–22, Mar. 1, 1964, ISSN: 0019-9958. DOI: 10.1016/S0019-9958(64)90223-2.
- [19] K. S. Fu, Ed., *Syntactic Pattern Recognition, Applications*, Communication and Cybernetics, Berlin Heidelberg: Springer-Verlag, 1977, ISBN: 978-3-642-66440-3. DOI: 10.1007/978-3-642-66438-0.
- [20] R. González and M. Thomason, "Syntactic pattern recognition, an introduction," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 1981. DOI: 10.1109/tpami.1981.4767110.
- [21] L. Miclet, "Syntactic and structural pattern recognition: Theory and applications," in *Syntactic and Structural Pattern Recognition: Theory and Applications*, H. Bunke and A. Sanfeliu, Eds., World Scientific, 1990, pp. 237–290, ISBN: 978-9971-5-0566-0.
- [22] E. Gold, "Language identification in the limit," *Information and Control*, vol. 10, pp. 447–474, May 1, 1967. DOI: 10.1016/S0019-9958(67)91165-5.
- [23] L. Pitt, "Inductive inference, DFAs, and computational complexity," in *Analogical and Inductive Inference*, K. P. Jantke, Ed., ser. Lecture Notes in Computer Science, Berlin, Heidelberg: Springer, 1989, pp. 18–44, ISBN: 978-3-540-46798-4. DOI: 10.1007/3-540-51734-0_50.
- [24] B. A. Trakhtenbrot and I. M. Barzdin, *Finite automata: behavior and synthesis*, ser. Fundamental studies in computer science v. 1. Amsterdam, New York: North-Holland Pub. Co.; American Elsevier, 1973, 321 pp., ISBN: 978-0-444-10418-2.
- [25] K. J. Lang, "Random DFA's can be approximately learned from sparse uniform examples," in *Proceedings of the fifth annual workshop on Computational learning theory*, ser. COLT '92, New York, NY, USA: Association for Computing Machinery, Jul. 1, 1992, pp. 45–52, ISBN: 978-0-89791-497-0. DOI: 10.1145/130385.130390.
- [26] C. de la Higuera, J. Oncina, and E. Vidal, "Identification of DFA: Data-dependent versus data-independent algorithms," in *Grammatical Interference: Learning Syntax from Sentences*, L. Miclet and C. de la Higuera, Eds., ser. Lecture Notes in Computer Science, Berlin, Heidelberg: Springer, 1996, pp. 313–325, ISBN: 978-3-540-70678-6. DOI: 10.1007/BFb0033365.
- [27] H. I. Akram, C. de la Higuera, H. Xiao, and C. Eckert, "Grammatical inference algorithms in MATLAB," in *Grammatical Inference: Theoretical Results and Applications*, J. M. Sempere and P. García, Eds., ser. Lecture Notes in Computer Science, Berlin, Heidelberg: Springer, 2010, pp. 262–266, ISBN: 978-3-642-15488-1. DOI: 10.1007/978-3-642-15488-1_22.